

Dividing

划分树

引入

划分树是一种来解决区间第 k 大的一种数据结构，其常数、理解难度都要比主席树低很多。同时，划分树紧贴「第 k 大」，所以是一种基于排序的一种数据结构。

建议先学完 [主席树](#) 再看划分树哦

过程

建树

原数组:	1	5	6	3	8	4	4	2
第一层:	1	3	4	2	5	6	8	4
第二层:	1	2	3	4	5	4	6	8
(log n)第三层:	1	2	3	4	4	5	6	8

划分树的建树比较简单，但是相对于其他树来说比较复杂。

如图，每一层都有一个看似无序的数组。其实，每一个被红色标记的数字都是 **要分配到左儿子的**。而分配的规则是什么？就是与 **这一层的中位数** 做比较，如果小于等于中位数，则分到左边，否则分到右边。但是这里要注意一下：并不是严格的 **小于等于就分到左边，否则分到右边**。因为中位数可能有相同，而且与 n 的奇偶有一定关系。下面的代码展示会有一个巧妙的运用，大家可以参照代码。

我们不可能每一次都对每一层排序，这样子不说常数，就算是理论复杂度也过不去。我们想，找中位数，一次排序就够了。为什么？比如，我们求 n 的中位数，其实就是在排序完后的 $num[mid]$ 。

两个关键数组：

$tree[log(N),N]$: 也就是树，要存下所有的值，空间复杂度 $O(N \log N)$ 。 $toleft[log(N),n]$: 也就是每一层 $1 \sim i$ 进入左儿子的数量，这里需要理解一下，这是一个前缀和。

▼ 实现

```

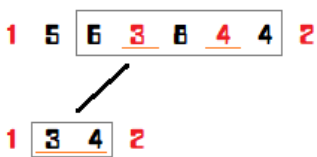
1 procedure Build(left,right,deep:longint); // left,right 表示区间左右端点,deep是第几层
2 var
3   i,mid,same,ls,rs,flag:longint; // 其中 flag 是用来平衡左右两边的数量的
4 begin
5   if left=right then exit; // 到底层了
6   mid:=(left+right) >> 1;
7   same:=mid-left+1;
8   for i:=left to right do
9     if tree[deep,i]<num[mid] then
10       dec(same);
11
12   ls:=left; // 分配到左儿子的第一个指针
13   rs:=mid+1; // 分配到右儿子的第一个指针
14   for i:=left to right do
15     begin
16       flag:=0;
17       if (tree[deep,i]<num[mid])or((tree[deep,i]=num[mid])and(same>0)) then // 分配到左边的条件
18         begin
19           flag:=1; tree[deep+1,ls]:=tree[deep,i]; inc(ls);
20           if tree[deep,i]=num[mid] then // 平衡左右个数
21             dec(same);
22         end
23       else
24         begin
25           tree[deep+1,rs]:=tree[deep,i]; inc(rs);
26         end;
27       toleft[deep,i]:=topleft[deep,i-1]+flag;
28     end;
29   Build(left,mid,deep+1); // 继续
30   Build(mid+1,right,deep+1);
31 end;

```

查询

那我们先扯一下主席树的内容。在用主席树求区间第 k 小的时候，我们以 $num[mid]$ 为基准，向左就向左，向右要减去向左的值，在划分树中也是这样的。

查询难理解的，在于 **区间缩小** 这种东西。下图，查询的是 $k=4$ 到，那么下一层就只需要查询 $k=1$ 到 4 了。当然，我们定义 l 为缩小后的区间（目标区间）， r 还是所在节点的区间。那为什么要标出目标区间呢？因为那是 **判定答案在左边还是右边的基准**。



▼ 实现

```

1 function Query(left,right,k,l,r,deep:longint):longint;
2 var
3   mid,x,y,cnt,rx,ry:longint;
4 begin
5   if left=right then // 写成 l=r 也无妨,因为目标区间也一定有答案
6     exit(tree[deep,left]);
7   mid:=(l+r) >> 1;
8   x:=topleft[deep,left-1]-topleft[deep,l-1]; // l 到 left 的去左儿子的个数
9   y:=topleft[deep,right]-topleft[deep,l-1]; // l 到 right 的去左儿子的个数
10  ry:=right-l-y; rx:=left-l-x; // ry 是 l 到 right 去右儿子的个数,rx 则是 l 到 left 去右儿子的个数
11  cnt:=y-x; // left 到 right 左儿子的个数
12  if cnt>=k then // 主席树常识啦
13    Query:=Query(l+x,l+y-1,k,l,mid,deep+1) // l+x 就是缩小左边界,l+y-1 就是缩小右区间。对于上图来说,就是把节点 1 和 2 放弃了。
14  else
15    Query:=Query(mid+rx+1,mid+ry+1,k-cnt,mid+1,r,deep+1); // 同样是缩小区间,只不过变成了右边而已。注意要将 k 减去 cnt。
16 end;

```

性质

时间复杂度：一次查询只需要，次询问，就是。

空间复杂度：只需要存储个数字。

亲测结果：主席树：、划分树：。（非递归，常数比较小）

划分树的应用

例题：[Luogu P3157\[CQOI2011\] 动态逆序对](#)

题意简述：给定一个个元素的排列（），有 m 次询问（），每次删去排列中的一个数，求删去这个数之后排列的逆序对个数。

这题可以使用 CDQ 在的时间及的空间内解决，并且 CDQ 的常数也很优秀。

如果这道题改为强制在线，则一般使用树状数组 + 主席树的树套树解法解决，时间复杂度为，空间复杂度为，常数略大，同样可以过此题。

而使用划分树的话就可以在的时间及的空间内在线解决本题，同时常数也比树套树解法少很多。（大致与 CDQ 相当。）

注意：为了编程实现方便，本文依照位置的中间值将大数组划分为两个小数组，即下文中的划分树相当于是归并排序的过程，而非快速排序的过程。最顶层的大数组为有序数组，最底层为原数组。

对于每一个划分树中的节点，我们称他为右节点当且仅当他在下一层会被划分到右孩子，即原数组中位置比较靠后的那些数，相似的可以定义左节点。如果在建树的过程中将最顶层排为有序的，类似于归并排序求逆序对，可以发现一个数组的逆序对个数就是在每个左节点之前的右节点的个树和。

再考虑删除操作。删除一个左节点会将整个数组的逆序对减少在他之前右结点的个数，而删除一个右节点会减少在他之后的左节点个数。那么可以考虑每次动态维护「每一个左节点之前的右结点数」和「每一个右节点之后的左节点个数」。这可以使用树状数组简单维护。

需要注意的是，在使用树状数组维护时只能计算在划分树中同一块内的贡献，而不能跳出块。对于树状数组来说有一个较为巧妙的处理方式。

考虑划分树上每一块的下标范围肯定为的形式，列举如下（由于代码中不会涉及到划分树最底层的处理，因此只枚举到倒数第二层）：

```
1 [0001 0010] [0011 0100] [0101 0110] [0111 1000] [1001 1010] [1011 1100] [1101 1110] [1111 10000] lev=1
2 [0001 0010 0011 0100] [0101 0110 0111 1000] [1001 1010 1011 1100] [1101 1110 1111 10000] lev=2
3 [0001 0010 0011 0100 0101 0110 0111 1000] [1001 1010 1011 1100 1101 1110 1111 10000] lev=3
4 [0001 0010 0011 0100 0101 0110 0111 1000 1001 1010 1011 1100 1101 1110 1111 10000] lev=4
```

回忆一下树状数组的原理，在向上跳的时候，我们每次 $x += \text{lowbit}(x)$ 。如果在向上跳的时候可以保证不跳出块，就可以保证只会影响到块内元素的值。向上查询也类似。

而如果要在向上跳的同时保证不跳出块，只需要保证在跳的时候满足即可。

而向下跳则是完全不同的处理方式。每一块的下标如果使用 0-index 表示的话，即为的形式。那么，只需将某一个下标的值右位移 k ，即可得出它在哪一块中。在向下跳的时候时刻判断是否跳出块即可。

需要注意的是，按这一方法实现的树状数组会访问到的最大下标是距离 n 最近的 2 的整次幂，因此数组下标不能开 n 。

由于需要在层修改，在第层修改的时间复杂度为，最终时间复杂度即为。

附代码：

```
1 #include <LAlgorithm>
2 #include <LTcstring>
```

```

3 #include <LTIostream>
4 #if defined(_MSC_VER) && !defined(__clang__)
5 #include <LTimmintrin.h>
6 #define __builtin_clz_lzcnt_u32
7 #endif
8 using namespace std;
9 using lld = long long;
10
11 int ti[2][17][131073];
12
13 int lowbit(int x) { return x & (-x); }
14
15 void fixup(int k, int x) {
16     for (; lowbit(x) < 1 << k; x += lowbit(x)) {
17         ++ti[0][k - 1][x];
18     }
19     ++ti[0][k - 1][x];
20 }
21
22 void fixdown(int k, int x) {
23     for (; ((x ^ lowbit(x)) - 1) >> k == (x - 1) >> k; x ^= lowbit(x)) {
24         ++ti[1][k - 1][x];
25     }
26     ++ti[1][k - 1][x];
27 }
28
29 int queryup(int k, int x) {
30     int res = 0;
31     for (; lowbit(x) < 1 << k; x += lowbit(x)) {
32         res += ti[1][k - 1][x];
33     }
34     return res + ti[1][k - 1][x];
35 }
36
37 int querydown(int k, int x) {
38     int res = 0;
39     for (; ((x ^ lowbit(x)) - 1) >> k == (x - 1) >> k; x ^= lowbit(x)) {
40         res += ti[0][k - 1][x];
41     }
42     return res + ti[0][k - 1][x];
43 }
44
45 int ai[100005];
46 int tx[100005];
47
48 lld mgx(int* npi, int* nai, int* rxi, int* lxi, int al, int ar, int bl,
49         int br) {
50     int rx = 1;
51     int lx = ar - al;
52     lld res = 0;
53     for (; al != ar || bl != br; ++npi, ++nai, ++rx, ++lxi) {
54         if (al != ar && (bl == br || tx[al] < tx[bl])) {
55             --lx;
56             *rx = rx;
57             *nai = tx[al];
58             *npi = al;
59             ++al;
60         } else {
61             ++rx;
62             *lxi = lx;
63             res += ar - al;
64             *nai = tx[bl];
65             *npi = bl;
66             ++bl;
67         }
68     }
69     return res;
70 }
71
72 int npi[1700005];

```

```

73 int nri[1700005];
74 int nli[1700005];
75
76 int main() {
77     cin.tie(nullptr)->sync_with_stdio(false);
78     int n, m;
79     cin >> n >> m;
80     for (int i = 1; i <= n; ++i) cin >> ai[i];
81
82     if (n == 1) {
83         for (int i = 1; i <= m; ++i) cout << "0\n";
84         return 0;
85     }
86
87     const int logn = 31 - __builtin_clz(n - 1);
88
89     lld ans = 0;
90     for (int i = logn; i >= 0; --i) {
91         memcpy(tx, ai, (n + 1) * sizeof(int));
92         for (int j = 1; j <= n; j += 1 << (logn - i + 1)) {
93             ans += mgx(npi + n * i + j, ai + j, nri + n * i + j, nli + n * i + j, j,
94                 min(n + 1, j + (1 << (logn - i))),
95                 min(n + 1, j + (1 << (logn - i))),
96                 min(n + 1, j + (1 << (logn - i + 1))));
97         }
98     }
99     cout << ans << '\n';
100
101     for (int asdf = 1; asdf < m; ++asdf) {
102         int x;
103         cin >> x;
104         for (int i = 0, p = 0; i <= logn; ++i, p += n) {
105             if (nri[p + x]) {
106                 ans -= nri[p + x] - querydown(logn - i + 1, x) - 1;
107                 fixdown(logn - i + 1, x);
108             } else {
109                 ans -= nli[p + x] - queryup(logn - i + 1, x);
110                 fixup(logn - i + 1, x);
111             }
112             x = npi[p + x];
113         }
114         cout << ans << '\n';
115     }
116 }

```

后记

参考博文 [:传送门](#)。

本页面最近更新：2024/12/11 21:43:19, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [Xarfa](#), [aofall](#), [CCXXXI](#), [CoelacanthusHex](#), [countercurrent-time](#), [danni](#), [Early0v0](#), [Enter-tainer](#), [FinParker](#), [H-J-Granger](#), [iamtwz](#), [Ir1d](#), [kenlig](#), [ksyx](#), [Marcythm](#), [NachtgeistW](#), [opsiff](#), [orzAtalod](#), [Persdre](#), [shuzhouliu](#), [sshwy](#), [StudyingFather](#), [SukkaW](#), [Tiphereth-A](#), [william-song-shy](#), [ZsgsDesign](#), [zyouxam](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用